

Module 13 - Fine-Tuned Language Model Admissions Classifier

Software Concepts | Johns Hopkins University

Student: Amr Mansour **JHED ID:** amanso8

This write-up documents the complete Module 13 workflow: preprocessing the admissions data, constructing unified language-model inputs, splitting the data, fine-tuning DistilBERT, evaluating the held-out test set, saving and reloading the model, deploying inference through Flask, comparing the result with the Module 12 NumPy neural network, and reflecting on limitations and ethics.

Final result	Value
Original / classification rows	14,188 / 12,712
Model	distilbert-base-uncased
Training / held-out test	10,169 / 2,543
Best checkpoint	Epoch 1
Accuracy	0.7393 (73.93%)
Precision / Recall / F1	0.7630 / 0.6200 / 0.6841
Module 12 accuracy	72.28%
Accuracy improvement	+1.65 percentage points

1. Load and Prepare the Applicant Dataset

The source dataset contained **14,188** applicant records. Duplicate checking by entry URL removed zero rows. Waitlisted records were excluded because the assignment uses binary classification. The final classification dataset therefore contained **12,712** records: **5,789 Accepted** and **6,923 Rejected**.

Feature type	Fields used
Text-based	program_name, university
Structured / non-text	degree, student_type, gpa, gre_score, gre_v_score, gre_aw, start_term

The target was encoded as **1 = Accepted** and **0 = Rejected**. Numeric fields were converted safely and missing values were handled consistently. The comments column contained no usable values and was therefore not included as a training feature.

2. Convert Each Applicant into a Unified Model Input

Each applicant was converted to one text sequence containing both textual and structured fields. Missing fields were represented as **Unknown**. The target label and applicant status were not included in the model input.

Exact input template

Program: <program_name>. University: <university>. Degree: <degree>. Citizenship: <student_type>. GPA: <gpa>. GRE: <gre_score>. GRE Verbal: <gre_v_score>. GRE Analytical Writing: <gre_aw>. Term: <start_term>.

Example: Program: Global Health. University: Meharry Medical College. Degree: PhD. Citizenship: American. GPA: 3.9. GRE: Unknown. GRE Verbal: Unknown. GRE Analytical Writing: Unknown. Term: Fall 2026.

3. Split the Data into Training and Testing Sets

The dataset was split with **test_size=0.20**, **random_state=42**, shuffling, and stratification on the binary label. This preserved the Accepted/Rejected balance and ensured evaluation occurred on applicants not used for training.

Set	Accepted	Rejected	Total
Training	4,631 (45.54%)	5,538 (54.46%)	10,169
Held-out test	1,158 (45.54%)	1,385 (54.46%)	2,543

4. Fine-Tune a Pretrained PyTorch Language Model

The selected pretrained model was **distilbert-base-uncased**. DistilBERT was selected because it supports sequence classification while being smaller and faster than full BERT. The matching DistilBERT tokenizer was used with truncation and fixed-length padding.

Training configuration	Value
Model / tokenizer	distilbert-base-uncased
Maximum sequence length	256
Batch size	8
Epochs	2
Learning rate	2e-5
Optimizer	AdamW
Device	CPU

Epoch	Train loss	Test loss	Test accuracy
1	0.5774	0.5422	73.89%
2	0.5365	0.5383	73.10%

Epoch 1 was retained as the best checkpoint. Its recorded test accuracy was 73.89% and test loss was 0.5422. The complete two-epoch CPU run took approximately 306 minutes. The best checkpoint was written to **saved_model** during training.

5. Evaluate the Final Model

The saved best checkpoint was reloaded and evaluated against the reconstructed held-out test set of 2,543 applicants. The required classification metrics are shown below.

Metric	Final value
Accuracy	0.7393 (73.93%)
Precision	0.7630
Recall	0.6200
F1 score	0.6841
Majority-class baseline	0.5446 (54.46%)

Confusion matrix - rows are actual classes; columns are predicted classes

Actual / Predicted	Rejected	Accepted
Rejected	1,162	223
Accepted	440	718

The confusion matrix contains 1,162 true negatives, 223 false positives, 440 false negatives, and 718 true positives. The model predicted Rejected for 63.00% of the test set, while the actual Rejected share was 54.46%, suggesting some tendency toward the Rejected class.

The model substantially exceeds the majority-class baseline, but the result should not be treated as a realistic admissions decision model. Important admissions variables are absent and GRE-related fields are highly sparse.

6. Save and Reload the Trained Model

The best model checkpoint was persisted so that inference could occur without retraining. The saved directory contains the fine-tuned weights, model configuration, tokenizer state, and metadata describing the training configuration, label mapping, fields, and input template.

Saved artifact	Purpose
model.safetensors	Fine-tuned DistilBERT weights
config.json	Sequence-classification model configuration
tokenizer.json / tokenizer_config.json	Saved tokenizer and tokenizer configuration
metadata.json	Preprocessing template, label mapping, hyperparameters, and best-checkpoint information

Reload verification passed successfully. The tokenizer and model were loaded locally from **saved_model**, placed in evaluation mode, and used for post-reload inference without retraining.

7. Build the "Will You Get In?" Webpage

A new Flask route, **/will-you-get-in**, was added to the existing application. The page collects program, university, term, degree type, citizenship, GPA, GRE, GRE Verbal, GRE Analytical Writing, and an optional comments/applicant-statement field. Missing optional values are converted to the same Unknown representation used during model preparation.

The comments field is displayed for the webpage exercise but is intentionally not passed into the model because the training dataset contained no usable comments. This prevents an inference-time feature mismatch. The saved DistilBERT model is loaded once by the Flask application and is not retrained for requests.

Verified webpage example	Value
Program / university	Computer Science / Johns Hopkins University
Degree / citizenship	Masters / American
GPA / GRE / Verbal / AW	3.90 / 325 / 160 / 4.5
Prediction	Accepted
Accepted probability	80.78%
Rejected probability	19.22%

The webpage clearly states that the result is generated by a class-project model trained on historical, scraped, self-reported admissions data and is not an actual university admissions decision. Blank-page and completed-prediction screenshots were captured as submission evidence.

8. Comparison with the Earlier Two-Layer Neural Network

The Module 12 model was a manually implemented two-layer NumPy neural network using six structured features: GPA, GRE, GRE Verbal, GRE Analytical Writing, degree type, and citizenship. Module 13 uses a pretrained transformer and a unified textual representation that also incorporates program and university information.

Comparison	Module 12	Module 13
Model	Two-layer NumPy NN	Fine-tuned DistilBERT
Input representation	Six structured features	Text + structured fields as unified text
Test accuracy	72.28%	73.93%
Pretrained knowledge	No	Yes
Deployment complexity	Low	Higher

The transformer improved accuracy by **1.65 percentage points**. Program and university information appear to add some predictive value, although the experiment cannot isolate how much of the improvement comes from those fields versus the pretrained representation. The lack of usable comments also means the project cannot test whether applicant statement text would improve results.

The transformer is more flexible but also more computationally expensive and operationally fragile. It requires tokenization, PyTorch and Transformers dependencies, a roughly 268 MB model artifact, and substantially longer CPU training. Given the modest accuracy improvement, the added complexity alone would not be justified for a real production admissions system without richer data and a clearer practical benefit.

9. Reflection on Limitations and Ethics

Fine-tuning a pretrained language model is useful because it adapts an existing representation to a specific task without training a large model from scratch. In this project, it also allowed program and university information to be combined naturally with structured applicant attributes. The resulting model was more flexible than the earlier NumPy network, although the improvement in accuracy was modest.

The most important limitation is the underlying data. The admissions records are self-reported and may not represent the full applicant population. People who choose to report results may differ systematically from those who do not, creating selection bias. The data can also contain reporting errors, inconsistent terminology, substantial missingness, and historical patterns that are not appropriate predictors of future admissions decisions.

Many factors used in real graduate admissions are absent: recommendation letters, research experience, publications, statement quality, faculty fit, interviews, program capacity, funding, and the overall strength and context of an application. Therefore, an output such as an 80% model score must not be interpreted as an 80% real-world chance of admission. It is only the classifier's relative confidence based on patterns in this limited dataset.

There are also significant ethical concerns. Historical data can encode institutional, geographic, social, and other biases. A transformer can learn associations involving universities, programs, citizenship categories, or other attributes without understanding whether those relationships are fair, appropriate, or causal. Using such a model for real admissions decisions could reproduce or amplify existing inequalities. For that reason, this model should not be used to make or recommend actual admissions decisions.

The public-facing page is nevertheless educationally valuable because it demonstrates the complete machine-learning lifecycle while making the model's limitations visible. Predictions should be presented as experimental rather than authoritative; uncertainty and limitations should be stated clearly; and users should be told explicitly that the result is a class-project prediction based on incomplete, self-reported data.

Conclusion

Module 13 successfully extended the earlier hand-built classifier into a complete pretrained transformer workflow: data preparation, unified model input construction, stratified train/test splitting, DistilBERT fine-tuning, held-out evaluation, checkpoint persistence, post-reload inference, and Flask deployment. The final model achieved **73.93% accuracy, 0.7630 precision, 0.6200 recall, and 0.6841 F1**. It outperformed the 54.46% majority baseline and the Module 12 network, while also demonstrating why technical performance alone is not enough to justify automated high-stakes decision-making.